プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

目次

- テーマ ブロック崩し1
  - ルール確認
  - 必要なフィールド、メソッドの検討
  - Gameクラスの実装

今後グロック崩しゲームの開発を実施(3回を予定)

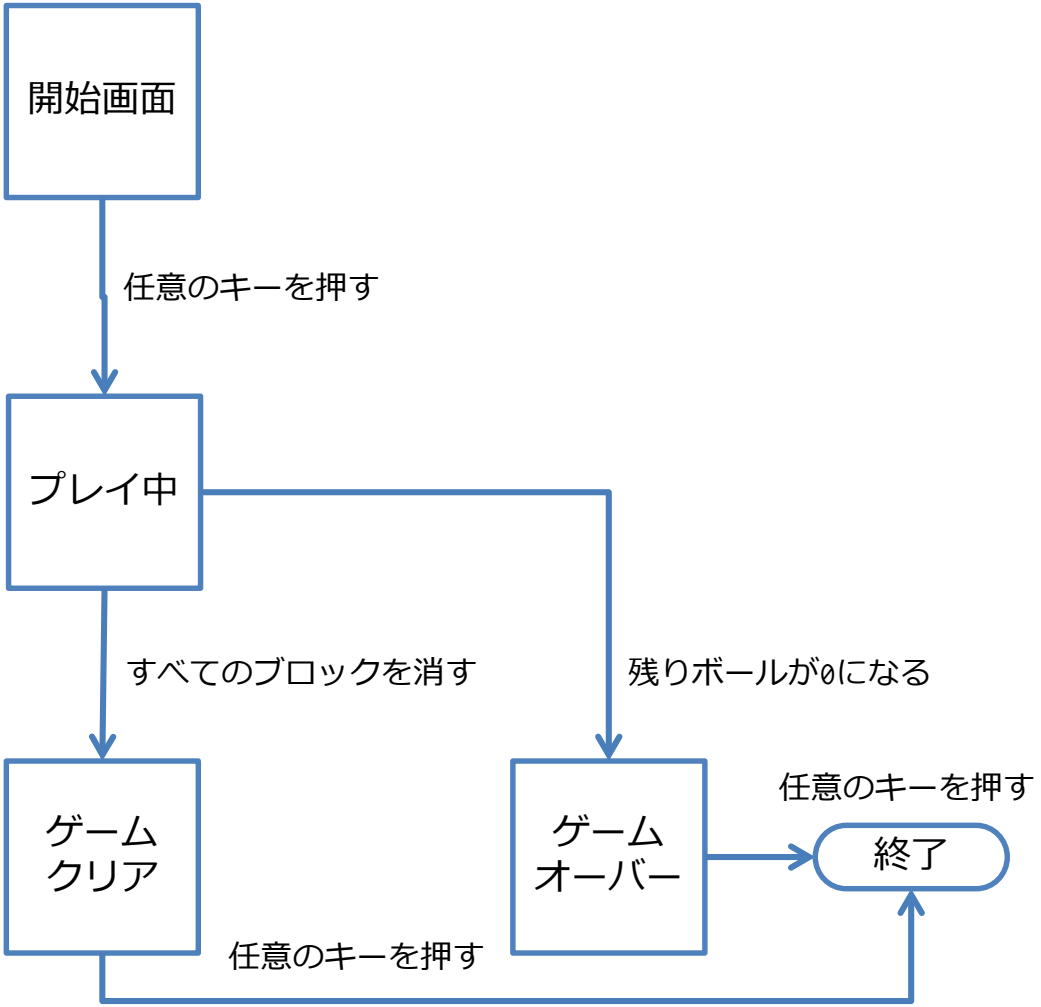
開発したプログラムをKadai_12として提出

ルール確認

- ボールが衝突するとブロックは消える
- 反射板を操作してボールを反射させる
- ボールを3個持つ、ブロックは？個配置
- ボールを画面下に落とすと1つボールが消える
- ブロックをすべて消すとゲームクリア
- ボールを3つ失うとゲームオーバー

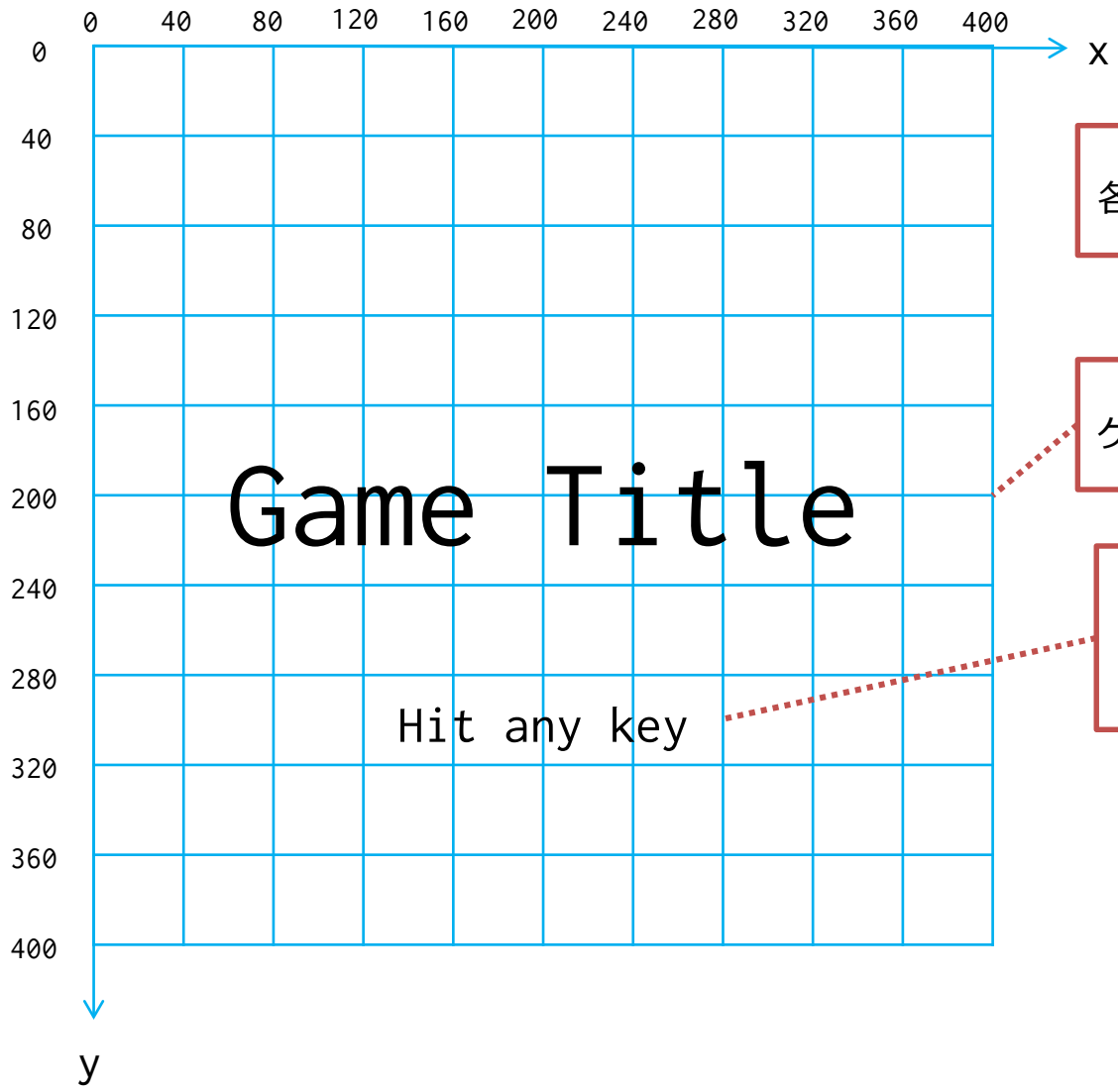
他に必要なルールはないか検討

画面遷移


開始画面
任意のキーを押す
プレイ中
すべてのブロックを消す
残りボールが0になる
ゲーム
クリア
ゲーム
オーバー
任意のキーを押す
終了
任意のキーを押す

開始画面

実行ウィンドウ
400 x 400

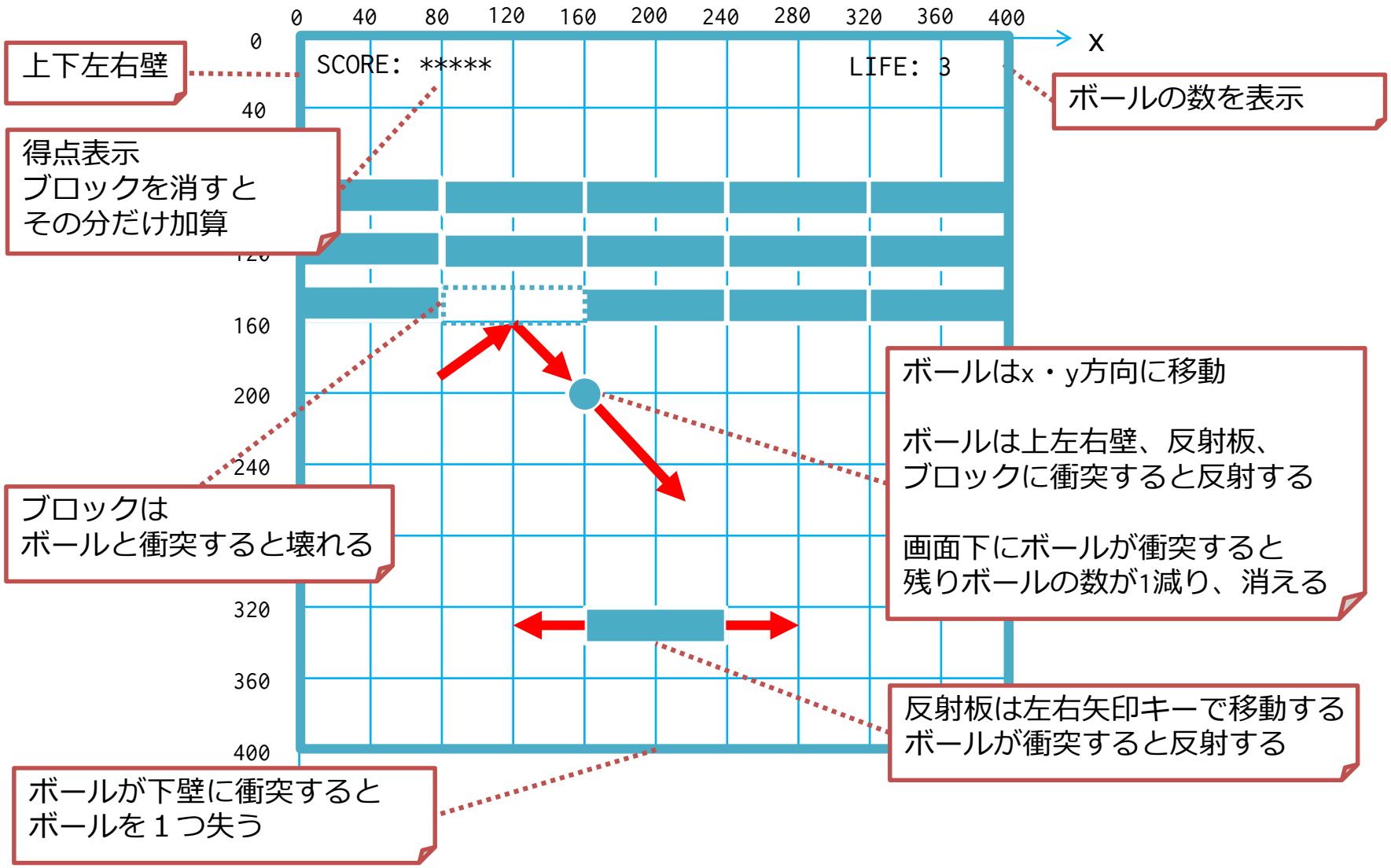


背景は
各自のアイディアで

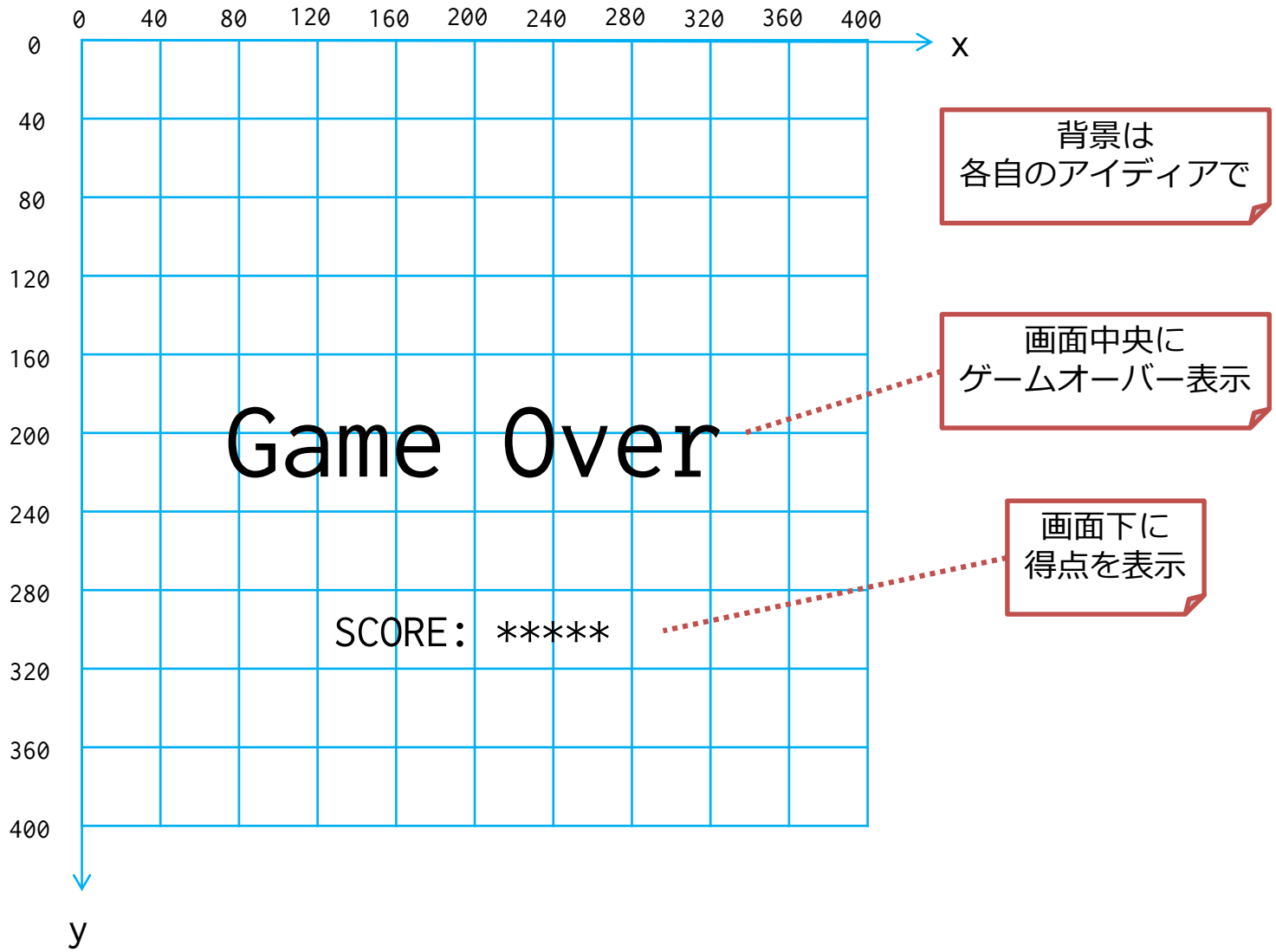
画面中央に
ゲームタイトル表示

画面下に
次の画面に移る
アクションを表示

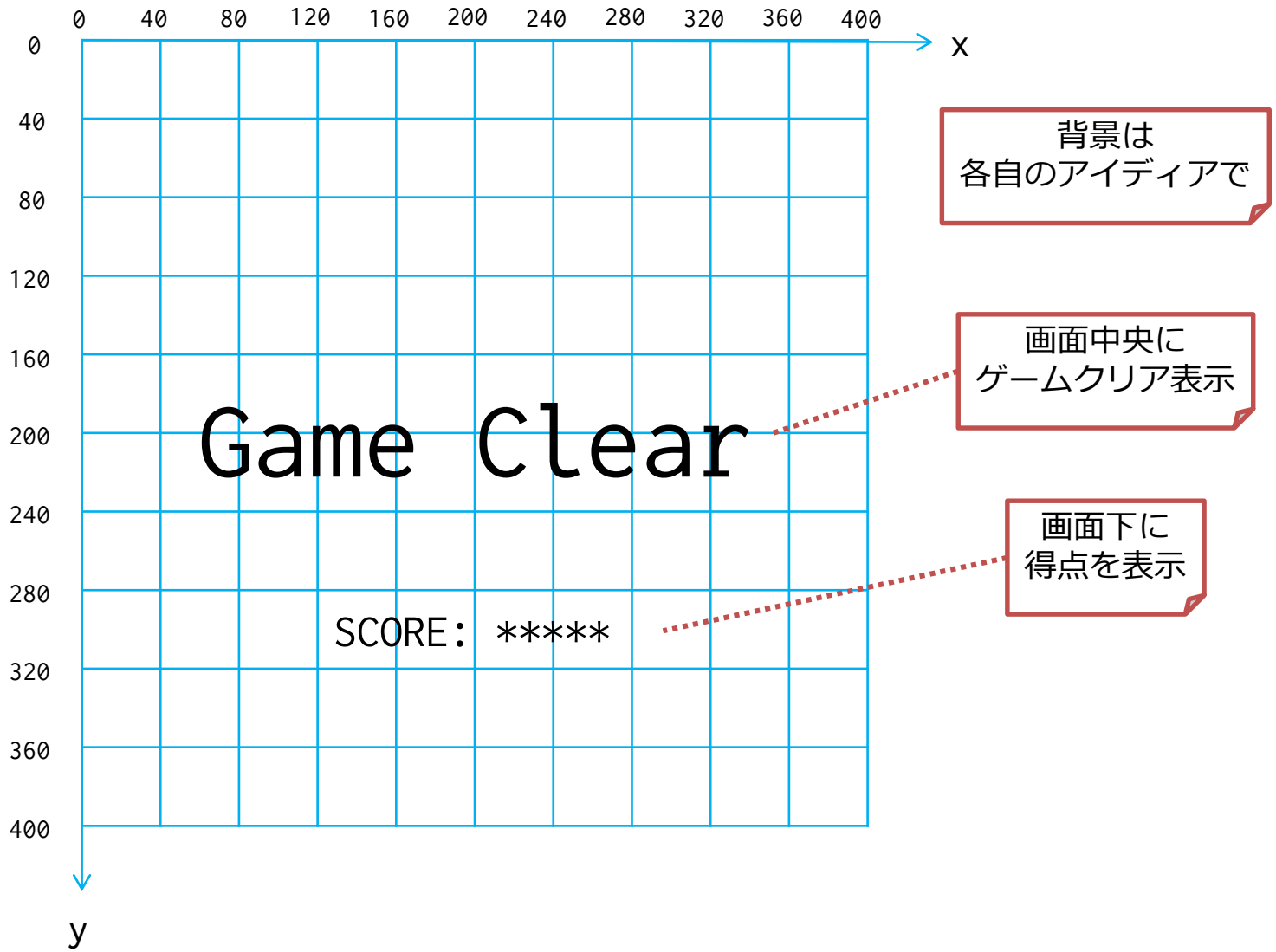
プレイ中の画面


0 40 80 120 160 200 240 280 320 360 400
x
0 40 160 200 240 320 360 400
SCORE: *****
LIFE: 3
上下左右壁
ボールの数を表示
得点表示
ブロックを消すと
その分だけ加算
ボールはx・y方向に移動
ボールは上左右壁、反射板、
ブロックに衝突すると反射する
画面下にボールが衝突すると
残りボールの数が1減り、消える
ブロックは
ボールと衝突すると壊れる
反射板は左右矢印キーで移動する
ボールが衝突すると反射する
ボールが下壁に衝突すると
ボールを１つ失う

ゲームオーバーの画面


0
40
80
120
160
200
240
280
320
360
400
x
y
Game Over
SCORE: *****
背景は
各自のアイディアで
画面中央に
ゲームオーバー表示
画面下に
得点を表示

ゲームクリアの画面


0
40
80
120
160
200
240
280
320
360
400
x
y
Game Clear
SCORE: *****
背景は
各自のアイディアで
画面中央に
ゲームクリア表示
画面下に
得点を表示

Gameクラス

- フィールド
  - ボールの保有数
  - ブロックの数
  - 反射板
  - ボール
  - 上壁、左壁、右壁
  - ブロック（配列）
- メソッド
  - クリアしたか
  - ゲームオーバーしたか
  - ボールを落としたか
  - ボールを1つ消す
  - 実行する

他に必要なフィールド・メソッドはないか検討

フィールド

- ボールの保有数　`int life;`
- ブロックの数　`int blockNum;`
- 反射板　`Bar bar;`
- ボール　`Ball ball;`
- 上壁、左壁、右壁、下壁

  `Wall topWall, leftWall, rightWall, bottomWall;`
- ブロック（配列）

  `Block[] blocks = new Block[blockNum];`

メソッド

- クリアしたか　`boolean isWin()`
  - ブロックの数が0以下
- ゲームオーバーしたか　`boolean isOver()`
  - ボールの数が0以下
- ボールを落としたか　`boolean isDrop()`
  - ボールと下壁が衝突したか
- ボールを1つ消す　`void removeBall()`
- 実行する　`void run()`
  - ブロック崩しの一連の処理

今回の作業

- Gameクラス追加
- これまでのプログラムの修正

Gameクラス①

```
class Game {

  //フィールド
  int life = 3;                                                    // ボールの数(初期値3)
  int blockNum = 5;                                                // ブロックの数(初期値5、今後修正する)

  Ball ball;                                                       // ボール
  Bar bar;                                                         // 反射板
  Wall topWall, bottomWall, leftWall, rightWall;                   // 上壁、下壁、左壁、右壁
  Block[] blocks = new Block[blockNum];                            // ブロック（配列）

  // コンストラクタ
  Game() {
    ball = new Ball(width/2, height/2, 0.5, 0.3);                  //ボールの初期位置(width/2, height/2)
    bar = new Bar(width/2, height*0.8);                            //反射板の初期位置(width/2, height*0.8)
    topWall    = new Wall(0, 0, width, 0);
    bottomWall = new Wall(0, height, width, 0);
    leftWall   = new Wall(0, 0, 0, height);
    rightWall  = new Wall(width, 0, 0, height);
    for (int i=0; i<blocks.length; i++) {
      blocks[i] = new Block(i*80, 80);
    }
  }
```

Gameクラス②

```
// クリアしたかどうか
boolean isWin() {
  if (blockNum <= 0) {            // (見えている)ブロックの数が０以下の場合
    return true;
  } else {
    return false;
  }
}
// ゲームオーバーしたかどうか
boolean isOver() {
  if (life <= 0) {                //ボールの数が０以下の場合
    return true;
  } else {
    return false;
  }
}
// ボールを落としたかどうか
boolean isDrop() {
  if (bottomWall.isHitTopSide(ball)) {        //下壁とボールが衝突した場合
    return true;
  } else {
    return false;
  }
}
```

Gameクラス③

```
// ボールを1つ減らす
void removeBall() {
  life--;                                                    // ボールの数を1減らす
  ball = null;                                               // ボールを消す
  ball = new Ball(width/2, height/2, 0.5, 0.3);              // ボールを初期位置に生成
}

// ゲームを実行する
void run() {
  background(0);

  if (!isWin() && !isOver()) { // ゲームクリアしていない かつ ゲームオーバーしていない場合

    ball.move();                    // ボールが移動する（座標のデータが更新する）
    bar.move();                     // 反射板が移動する（座標のデータが更新する）

    if (leftWall.isHitRightSide(ball) ||       // ボールと左壁の右側が衝突する または
      rightWall.isHitLeftSide(ball)) {         // ボールと右壁の左側が衝突する 場合
      ball.flipXdir();                                       // ボールをx方向に反転する
    }
    if (topWall.isHitBottomSide(ball)) {       // ボールと上壁の下側が衝突した場合
      ball.flipYdir();                                       // ボールをy方向に反転する
    }
```

Gameクラス④

```
if (isDrop()) {    // ボールが落ちたら
  removeBall();    // ボールを1個減らす
}

if (bar.isHitTopSide(ball) ||              // ボールと反射板の上側が衝突する または
      bar.isHitBottomSide(ball)) {         // ボールと反射板の下側が衝突する 場合
  ball.flipYdir();                                  // ボールをy方向に反転する
}
if (bar.isHitRightSide(ball) ||            // ボールと反射板の右側が衝突する または
      bar.isHitLeftSide(ball)) {                    // ボールと反射板の左側が衝突する 場合
  ball.flipXdir();                                  // ボールをx方向に反転する
}
if (bar.isHitCorner(ball)) {               // ボールと反射板の各頂点が衝突する 場合
  ball.flipXdir();                                  // ボールをx方向に反転する
  ball.flipYdir();                                  // ボールをy方向に反転する
}

for (int i=0; i<blocks.length; i++) {  // ブロックの数だけ繰り返す
  if (blocks[i].isHitTopSide(ball) ||      // ボールとi番目のブロックの上側が衝突する または
      blocks[i].isHitBottomSide(ball)) {   // ボールとi番目のブロックの下側が衝突する 場合
    ball.flipYdir();                                // ボールをy方向に反転する
    blocks[i].remove();                    // i番目のブロックを消す
    blockNum--;                            // ブロックの数を1減らす
  }
```

Gameクラス⑤

```
        if (blocks[i].isHitRightSide(ball) || // ボールとi番目のブロックの右側が衝突するまたは
            blocks[i].isHitLeftSide(ball)) {     // ボールとi番目のブロックの左側が衝突する 場合
          ball.flipXdir();                       // ボールをx方向に反転する
          blocks[i].remove();                    // i番目のブロックを消す
          blockNum--;                            // ブロックの数を1減らす
        }
        if (blocks[i].isHitCorner(ball)) {       // ボールとi番目のブロックの各頂点が衝突する場合
          ball.flipXdir();                       // ボールをx方向に反転する
          ball.flipYdir();                       // ボールをy方向に反転する
          blocks[i].remove();                    // i番目のブロックを消す
          blockNum--;                            // ブロックの数を1減らす
        }
      }
    } else {             // それ以外の場合（ゲームオーバー または ゲームクリア の場合）
      exit();            // プログラム終了（現時点の対応、あとで修正する）
    }

    ball.display();                              // ボールを表示
    bar.display();                               // 反射板を表示
    for (int i=0; i<blocks.length; i++) {        // ブロックの数だけ繰り返す
      blocks[i].display();                       // i番目のブロックを表示
    }
  }
}
```

メインタブ

```
Game game;

void setup() {
  size(400, 400);
  ellipseMode(RADIUS);
  game = new Game();
}

void draw() {
  game.run();
}
```

これまでに作成した
Ballクラス、
Barクラス、
Blockクラス、
Rectクラス、
Wallクラスは
そのまま利用する